Mohammed Qureshi (101286283)

Predictive Load Balancer - Final Report - COMP 3106

1. Introduction

1.1 Background and Motivation

Load balancing is a fundamental component of modern distributed systems, responsible for distributing incoming network traffic across multiple backend servers to ensure reliability, minimize response times, and maximize resource utilization. Traditional load balancers employ reactive strategies such as round-robin distribution or least connections routing, responding to system state only after conditions change. This reactive paradigm presents significant limitations in production environments where traffic patterns exhibit predictable temporal characteristics.

During my summer 2025 software engineering internship working with trading systems, I observed firsthand how traditional reactive load balancing methods struggled during traffic spikes. The system would detect overload conditions only after latency had already degraded, leading to cascade failures and poor user experience. This experience, combined with my coursework in distributed systems (COMP 4001), motivated the development of a proactive load balancing system that anticipates traffic patterns rather than merely reacting to them. Although predictive scaling does exist in the real world (often mixed with reactive + predictive for the best possible user experience), this project aims to provide a sample demo on pure predictive scaling techniques.

1.2 Project Objectives

This project aims to bridge practical infrastructure engineering with artificial intelligence techniques by building a predictive load balancing system that addresses three core challenges. First, the system uses time-series forecasting to predict traffic patterns 1-5 minutes ahead, enabling proactive resource scaling before demand increases. Second, it implements intelligent request routing using search algorithms (A*) that consider multiple factors including server load, cache locality, and health status. Third, it applies reinforcement learning to learn optimal scaling policies that balance competing objectives of latency, cost, and reliability.

1.3 Related Prior Work

Commercial cloud providers implement auto-scaling based on threshold rules (e.g., scale when CPU exceeds 80%), but these reactive approaches cannot anticipate demand spikes. Academic research has explored predictive scaling using various machine learning techniques. It’s been demonstrated the effectiveness of time-series prediction for cloud resource management, while also applied reinforcement learning to auto-scaling decisions. Facebook's Prophet model (linear regression behind the scenes) has

been widely adopted for time-series forecasting due to its ability to capture daily and weekly seasonality patterns automatically.

This project integrates multiple AI techniques into a cohesive system: Prophet for traffic prediction, A* search for intelligent routing, and Q-learning for adaptive scaling. While each technique has been studied individually, their combination in a practical load balancing context represents a novel contribution suitable for demonstrating AI concepts in a systems engineering application.

2. Methods

2.1 Methods from Artificial Intelligence

2.1.1 Time-Series Prediction with Facebook Prophet

Prophet is an additive regression model developed by Facebook for time-series forecasting. The model decomposes the time series into trend, seasonality, and holiday components:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

Where $g(t)$ represents the trend function modeling non-periodic changes, $s(t)$ represents periodic changes (daily and weekly seasonality), $h(t)$ represents holiday effects, and $\varepsilon(t)$ represents the error term.

Prophet is particularly suited for this application because it:

- Automatically detects daily and weekly seasonality patterns common in web traffic
- Provides uncertainty intervals for predictions (critical for risk-aware scaling decisions)
- Handles missing data and outliers robustly
- Requires minimal hyperparameter tuning

The model predicts traffic volume for the next 1-5 minutes, with confidence intervals informing how aggressive or conservative scaling decisions should be.

2.1.2 Intelligent Routing with A* Search

A* is a best-first search algorithm that finds optimal paths by combining actual cost with heuristic estimates. In the routing context, the problem is formulated as selecting the optimal server for each incoming request. Since all servers are directly accessible (no multi-hop routing), the path cost $g(n)$ is zero, and the decision reduces to comparing heuristic values $h(n)$ for each server.

The custom heuristic function evaluates each server based on multiple weighted factors:

$$\mathbf{h(server) = w1*load + w2*queue + w3*health_penalty + w4*response_time - w5*cache_bonus}$$

Factor	Weight	Description
Load	10 points per active request	Penalizes busy servers
Queue	5 points per pending request	Accounts for waiting requests
Health Penalty	+1000 if unhealthy	Effectively removes unhealthy servers
Response Time	0.5 × avg_ms	Penalizes historically slow servers
Cache Bonus	−50 if cached	Rewards cache locality (50ms hit vs 200ms miss)

This multi-factor heuristic ensures requests are routed to servers that will provide the fastest response while maintaining load balance.

2.1.3 Adaptive Scaling with Q-Learning

Q-learning is a reinforcement learning algorithm that learns optimal action selection policies through trial and error. The agent learns a Q-function Q(s, a) that estimates the expected cumulative reward for taking action a in state s.

Q-value update: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma * \max Q(s',a') - Q(s,a)]$

Where $\alpha = 0.1$ (learning rate), $\gamma = 0.95$ (discount factor), r is immediate reward, and s' is the resulting state.

State Space (135 total states):

Component	Values	Description
Server count	1-5	Current number of active servers

Load level	Low/Medium/High	Based on thresholds at 30% and 70%
Prediction trend	Decreasing/Stable/Increasing	From Prophet forecasts
Latency status	OK/Warning/Critical	P95 latency thresholds at 100ms and 200ms

Action Space: Scale up, Scale down, Hold (with 60-second cooldown)

Condition	Reward	Purpose
Latency > 200ms	−10	Penalize critical latency
Latency 100-200ms	−5	Penalize warning latency
Each running server	−0.5	Encourage efficiency
Low latency + few servers	+5	Reward efficient operation
Any scaling action	−1	Discourage unnecessary changes

2.2 Dataset

The training data consists of NASA HTTP server logs from July 1995, a publicly available dataset containing 1,891,715 HTTP requests recorded over approximately one month. Each log entry follows the Apache Common Log Format.

Data Processing Pipeline:

Stage	Input	Output	Success Rate
Extraction	Raw Apache logs	Structured CSV	99.74% (1,886,705 entries)
Cleaning	Extracted CSV	Validated CSV	99.96% retention
Feature Engineering	Request-level	Minute-level	39,470 rows, 23 features

23 Engineered Features Across 5 Categories:

1. **Time Features (5):** hour, day_of_week, is_weekend, is_business_hours, time_of_day
2. **Aggregated Metrics (4):** request_count (target), total_bytes, avg_bytes, success_rate
3. **Rolling Windows (5):** 5/15/30/60-minute moving averages, 15-minute standard deviation
4. **Lag Features (6):** Traffic from 1, 5, 10, 15, 30, 60 minutes prior
5. **Statistical Features (3):** Rate of change, historical same-hour average, deviation from historical

2.3 Task Environment

The system runs as a Docker-orchestrated distributed environment with five containers:

Container	Technology	Purpose
Load Balancer	FastAPI	Prophet prediction, A* routing, Q-learning scaling
Backend 1-3	FastAPI	Application servers with LRU caching
PostgreSQL	PostgreSQL 14	Metrics storage

Latency Simulation: Cache hits return in ~50ms, cache misses in ~200ms (real backend work isn't implemented in this project, so when a request comes in a latency for backend work is simulated).

System Operation:

- Prediction service runs Prophet every 60 seconds
- A* router fetches server state every 5 seconds
- Q-learning agent uses greedy exploration (ε: 1.0 → 0.1)

2.4 Validation Strategy

Train/Test Split: 80/20 chronological (no shuffling to prevent data leakage since time series is sorted in increasing order)

- Training: First 22 days (~31,576 samples)
- Testing: Final 6 days (~7,894 samples)

Prediction Metrics: MAE, RMSE, R^2 (target: MAE < 15% of average traffic)

Routing Comparison: Controlled experiments comparing round-robin, greedy, and A* under identical traffic scenarios.

3. Results

3.1 Data Processing Results

Metric	Value
Raw log entries	1,891,715
Extraction success rate	99.74%
Failed entries	5,010 (0.26%)
Cleaning retention	99.96%
Final minute-level rows	39,470
Compression ratio	48×

3.2 Prediction Model Performance

Model	MAE (req/min)	RMSE (req/min)	R^2
Prophet	**11.23**	**14.79**	**0.659**
Ridge Regression	11.50	15.03	0.648
Linear Regression	11.50	15.03	0.648

Prophet's R^2 of 0.659 indicates the model explains 65.9% of traffic variance, successfully capturing daily patterns (high during business hours, low overnight) and weekly patterns (reduced weekend traffic). The remaining unexplained variance corresponds primarily to random burst traffic that is inherently unpredictable.

Visual analysis confirms Prophet smooths over individual traffic spikes while correctly tracking overall patterns desirable behavior for capacity planning where smooth predictions lead to stable scaling decisions.

3.3 Routing Algorithm Comparison

A* demonstrates the value of cache-aware request distribution. Example with request for /api/users:

Server 1: 2 active requests, has /api/users cached **Server 2:** 0 active requests, no cache

Score Calculation:

- Server 1: (2×10) + (100×0.5) − 50 = **20** ← Winner
- Server 2: (0×10) + (100×0.5) − 0 = **50**

The 50-point cache bonus outweighs the 20-point load penalty, correctly reflecting that a 50ms cache hit on a busier server beats a 200ms cache miss on an idle server. Round-robin ignores this optimization entirely.

3.4 Q-Learning Agent Behavior

The Q-learning agent successfully learns scaling policies through interaction. After training, the Q-table shows intuitive learned policies:

State	Best Action	Learned Q-Value
Low load, OK latency, 4 servers	Scale down	+3.2
High load, increasing trend, warning latency	Scale up	+4.1
Medium load, stable trend, OK latency	Hold	+1.0

The integration with Prophet predictions allows proactive scaling, initiating scale-up when Prophet predicts traffic increase even if current metrics are acceptable

4. Discussion

4.1 Key Technical Insights

Data Leakage Discovery: A critical issue discovered during development was data leakage in the feature engineering pipeline. Initial models achieved suspiciously perfect R^2 scores (approaching 1.0) because lag features inadvertently included current traffic when predicting current traffic. The fix required shifting all lag features by one additional period, when predicting traffic at time T, features must use only data from time T−1 and earlier.

Timestamp Handling: Prophet learns temporal patterns (daily/weekly cycles) rather than absolute timestamps, so patterns transfer across decades. The prediction service generates current timestamps for inference, using only temporal features rather than actual dates from the training set.

4.2 Limitations and Future Work

Limitation	Future Enhancement
Q-learning doesn't execute actual scaling	Implement Docker SDK integration for container orchestration
Hardcoded heuristic weights	Adaptive weight tuning based on observed behavior
Sequential request processing	Concurrent request handling

No visualization	Real-time WebSocket dashboard

4.3 Implications

This project demonstrates the practical application of multiple AI techniques to a real systems engineering problem. The combination of forecasting (Prophet), search (A*), and reinforcement learning (Q-learning) creates a system where each component addresses a different aspect of the load balancing challenge:

- **Prophet** provides foresight into traffic patterns
- **A*** optimizes individual routing decisions using multiple factors
- **Q-learning** learns capacity management policies balancing competing objectives

The modular architecture separates concerns effectively, predictions, routing, and scaling operate independently and can be improved or replaced individually.

5. References

[1] NASA-HTTP. (1995). Two months of HTTP requests to the NASA Kennedy Space Center WWW server. Internet Traffic Archive. http://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html

[2] Gong, Z., Gu, X., & Wilkes, J. (2010). PRESS: PRedictive Elastic ReSource Scaling for cloud systems. *Proceedings of the 2010 International Conference on Network and Service Management*.

[3] Roy, N., Dubey, A., & Gokhale, A. (2011). Efficient autoscaling in the cloud using predictive models for workload forecasting. *IEEE International Conference on Cloud Computing*.

6. Implementation

The complete implementation is available at: **https://github.com/2025F-COMP3106/project-group-101**

Repository Structure:

- scripts/ - Data pipeline (extraction, cleaning, feature engineering, model training)
- load_balancer/ - FastAPI load balancer with Prophet, A* routing, Q-learning
- backend/ - Backend server with LRU caching simulation
- models/ - Trained Prophet model (prophet_model.pkl)
- docker-compose.yml - Container orchestration configuration
- results/ - Model evaluation metrics and prediction plots